# BURSA TEKNİK

# ÜNİVERSİTESİ

## BİLGİSAYAR MÜHENDİSLİĞİ

## NESNEYE YONELIK PROGRAMLAMA

## PROJE ÖDEVİ

**HAZIRLAYAN**:

Abdul Matin HAQEQAT

No: 24360859224

Bilgisayar Mühendisliği Bölümü / Şube 1

**DANIŞMAN**:

ARŞ. GÖR. ZEYNEP BARUT

*2025–2026 BAHAR DÖNEMİ*

# ÖZET

Bu projede, Java programlama dili kullanılarak nesne yönelimli programlama prensiplerine dayalı bir banka otomasyon sistemi geliştirilmiştir. Sistem içerisinde müşteri oluşturma, hesap açma, para yatırma, para transferi, kredi kartı tanımlama ve borç ödeme işlemleri gerçekleştirilmektedir. Uygulama modüler bir yapıda tasarlanmış olup kalıtım, kapsülleme ve ArrayList yapıları etkin şekilde kullanılmıştır.

# 1. GİRİŞ

Bu projenin amacı, nesne yönelimli programlama prensiplerine uygun bir banka otomasyon sistemi geliştirmektir. Sistem; müşteri, banka personeli, banka hesapları ve kredi kartı gibi temel bankacılık yapılarını modellemektedir.

Program çalıştırıldığında kullanıcı tarafından müşteri oluşturulmakta, ardından hesap açma, para yatırma, hesaplar arasında para transferi yapma, kredi kartı oluşturma ve borç ödeme işlemleri gerçekleştirilmektedir. Ayrıca belirli şartlar sağlandığında hesap silme işlemi uygulanabilmektedir.

Programın genel çalışma akışı şu şekildedir:

I. müşteri oluşturulması

II. banka personeli oluşturulması

III. müşteri adına hesap açılması

IV. hesaba para yatırılması

V. hesaplar arasında para transferi yapılması

VI. kredi kartı tanımlanması

VII. kredi kartı borç ödeme işleminin yapılması

VIII. hesap silme işleminin gerçekleştirilmesi

GitHub Proje Kodu:

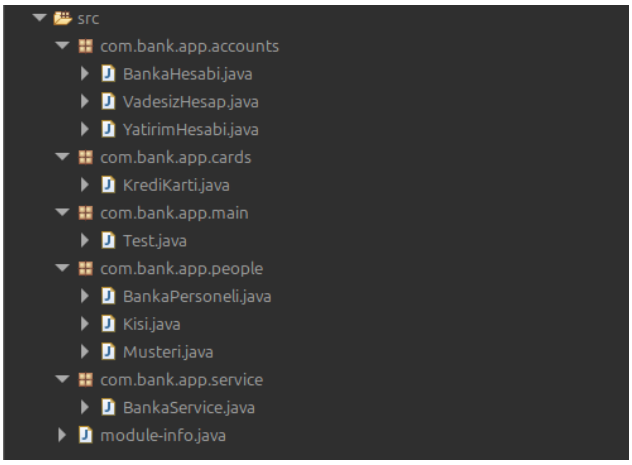
https://github.com/matinhaqeat/NYP_24360859224_AbdulMatinHAQEQAT

YouTube Proje Videosu:
https://youtu.be/PplxQKgngXI

# 2. TEKNİK DETAYLAR

## 2.1. Sistem Mimarisi ve Paket Yapısı

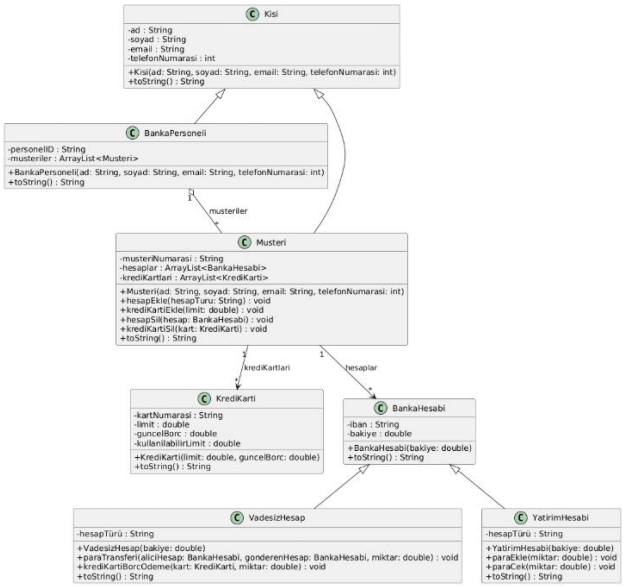
Sistem modüler bir yapıda geliştirilmiş olup sınıflar farklı paketler altında organize edilmiştir. Bu yapı kodun okunabilirliğini ve sürdürülebilirliğini artırmaktadır.




**Şekil 1.** Projenin Eclipse ortamındaki paket yapısı görünümü

## 2.2. UML Sınıf Diyagramı

Sistemdeki sınıflar arasındaki ilişkiler UML diyagramı ile modellenmiştir.



**Şekil 2.** Sınıflar arası inheritance ve association ilişkilerini gösteren UML diyagramı

## 2.3. Sınıf Yapıları

### 2.3.1. Kisi Sınıfı

Kisi sınıfı, müşteri ve banka personeli için ortak özellikleri içermektedir.

```java
package com.bank.app.people;

public class Kisi {
    //the variables:
    private String ad;
    private String soyad;
    private String email;
    private int telefonNumarasi;
```

**Şekil 3.** Kisi sınıfı tanımı (ad, soyad, email, telefonNumarasi)

### 2.3.2. Musteri Sınıfı

Müşteri sınıfı hesaplar ve kredi kartlarını yönetmektedir.

```java
public class Musteri extends Kisi{
    //Data fields:
    private String musteriNumarasi;
    private ArrayList<BankaHesabi> hesaplar;
    private ArrayList<KrediKarti> krediKartlari;

    //Constructor:
    public Musteri(String ad, String soyad, String email, int telefonNumarasi) {
        super(ad, soyad, email, telefonNumarasi);
        musteriNumarasi = "A" + (new Random().nextInt(90000000) + 10000000);
        hesaplar = new ArrayList<>();
        krediKartlari = new ArrayList<>();
    }
    //Methods:
    public void hesapEkle(String hesapTuru) {
    }
    public void krediKartiEkle(double limit) {
    }

    public ArrayList<KrediKarti> getKrediKartlari() {
        return this.krediKartlari;
    }
    public void hesapSil(BankaHesabi hesap) {
    }
    public void krediKartiSil(KrediKarti kart) {
    }
    public ArrayList<BankaHesabi> getHesaplar() {
        return hesaplar;
    }
    @Override
    public String toString() {
    }
```

**Şekil 4.** Musteri sınıfı ve hesap/kredi kartı listeleri

## 2.3.3. BankaPersoneli Sınıfı

Banka personeli müşterileri yönetmektedir.

```java
public class BankaPersoneli extends Kisi{
    //Data fields:
    Random rand = new Random();
    private String personeliID;
    private ArrayList<Musteri> musteriler = new ArrayList<>();

    //Constructor:
    public BankaPersoneli(String ad, String soyad, String email, int telefonNumarasi) {
    }

    //Getter function:
    public ArrayList<Musteri> getMusteri() {
    }

    //toString method;
    @Override
    public String toString() {
    }

}
```

**Şekil 5.** Banka personeli ve müşteri listesi yapısı

## 2.3.4. BankaHesabi ve Türevleri

Hesap işlemleri bu yapı üzerinden gerçekleştirilmiştir.

```java
package com.bank.app.accounts;
import java.util.Random;

public class BankaHesabi {
    //data fields:
    private String iban;
    private double bakiye;

    //constructor:
    public BankaHesabi(double bakiye) {
        this.bakiye = bakiye;

        //creating random IBAN:
        iban = "TR";
        Random rand = new Random();

        for(int i=0; i<24; i++) {
            char temp = (char) ('0' + rand.nextInt(10));
            iban += temp;
        }

    }
    //methods:
    @Override
    public String toString() {
        return "IBAN: " + iban + "\n" + "Bakiye: " + bakiye;
    }
    //Getters and Setters:
    public String getIban() {
        return iban;
    }
    public void setIban(String iban) {
        this.iban = iban;
    }
    public double getBakiye() {
        return bakiye;
    }
    public void setBakiye(double bakiye) {
        this.bakiye = bakiye;
    }


}
```

**Şekil 6.** Banka hesabı temel sınıfı ve IBAN/bakiye yapısı

## 2.3.5. KrediKarti Sınıfı

Kredi kartı limit ve borç yönetimi içerir.

```java
public class KrediKarti {
    //data Fields:
    private String kartNumarasi;
    private double limit;
    private double guncelBorc;
    private double kullanilabilirLimit;

    //Constructor:
    public KrediKarti (double limit, double guncelBorc) {
        kartNumarasi = String.valueOf(new Random().nextInt(90000000) + 10000000);
        this.limit = limit;
        this.guncelBorc = guncelBorc;
    }
    //Getters and Setters:
    public String  getKartNumarasi() {
        return kartNumarasi;
    }
    public double getLimit() {
        return limit;
    }
    public double getGuncelBorc() {
        return guncelBorc;
    }
    public double getKullanilabilirLimit() {
        return kullanilabilirLimit;
    }
    public void setLimit(double limit) {
        this.limit = limit;
    }
    public void setGuncelBorc(double guncelBorc) {
        this.guncelBorc = guncelBorc;
    }
    public void setKullanilabilirLimit(double kullanilabilirLimit) {
        this.kullanilabilirLimit = kullanilabilirLimit;
    }
    //toString method:
    public String toString() {
        return  "Kart Numarasi: " + kartNumarasi + "\n" +
                "Limit: " + limit + "\n" +
                "Guncel Borc: " + guncelBorc + "\n" +
                "Kullanilabilir Limit: " + (limit - guncelBorc);
    }

}
```

**Şekil 8.** Kredi kartı sınıfı (limit, borç, kullanılabilir limit)

## 2.4. Servis Katmanı

BankaService sınıfı tüm iş mantığını yönetmekte olup, müşteri işlemleri, hesap operasyonları ve kredi kartı işlemleri gibi tüm temel bankacılık süreçlerinin merkezi olarak kontrol edilmesini ve sistem içerisinde düzenli bir şekilde yürütülmesini sağlamaktadır.

```java
public class BankaService {
    //musteri olusturma:
    public Musteri musteriOlustur(String ad, String soyad, String email, int telefonNumarasi) {
        return new Musteri(ad,soyad,email,telefonNumarasi);
    }
    //yaratilan musteri icin hesap acma:
    public void hesapAc(Musteri musteri, String hesapTuru) {
        musteri.hesapEkle(hesapTuru);
    }
    //acilan hesapa para yatirma
    public void paraYatir(BankaHesabi hesap, double miktar) {
        hesap.setBakiye(hesap.getBakiye() + miktar);
    }
    //hesaplar icin para trasferi:
    public void paraTransferi(VadesizHesap gonderenHesap, BankaHesabi aliciHesap, double miktar) {
        gonderenHesap.paraTransferi(aliciHesap, gonderenHesap, miktar);
    }
    //kredikarti tanimlama:
    public void krediKartTanimla(Musteri musteri, double limit) {
        musteri.krediKartiEkle(limit);
    }
    //borc odeme:
    public void borcOde(VadesizHesap hesap, KrediKarti kart, double miktar) {
        hesap.kredikartiBorcOdeme(kart, miktar);
    }
    //hesapsilme:
    public void hesapSil(Musteri musteri, BankaHesabi hesap) {
        musteri.hesapSil(hesap);
    }
}
```

**Şekil 9.** Banka servis katmanı ve işlem metodları

## 2.5. Main Uygulama Akışı

Sistemin çalıştığı ana senaryo Main sınıfında gösterilmektedir.

```
Creating User...
Ad:Abdul Matin
Soyad:DEMIR
Email: demirabdulmatin@gmail.com
Telefon numarasi: 5514812
Musteri NO: A76435880



Banka Personeli olusturmaktadir...
Ad:Ali
Soyad:YILMAZ
Email: yilmazali123@gmail.com
Telefon numarasi: 5518929
Personel ID: ID887383



Hesap acilmaktadir...
Para yatirilmaktadir...
IBAN: TR441678382551068034093837
Bakiye: 1000.0
Hesap Turu: vadesizPara transferi gerceklestirilmektedir!
```

```
Para Gonderme:

Gonderen Hesap:
IBAN: TR441678382551068034093837
Bakiye: 0.0
Hesap Turu: vadesiz

Alinan Hesap:
IBAN: TR009522937475004287130196
Bakiye: 1000.0
Hesap Turu: vadesiz

Gonderilen Miktar: 1000



Kredi Kart tanimlanmaktadir...



alinan borc odenmektedir...
Kart Numarasi: 14318671
Limit: 10000.0
Guncel Borc: 400.0
Kullanilabilir Limit: 9600.0



Acilan hesap silinmektedir...
Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız.
```

**Şekil 10.** Caption: Programın çalıştırılması ve işlem çıktıları

# 3. SONUÇ

Bu projede nesne yönelimli programlama prensipleri kullanılarak bir banka otomasyon sistemi geliştirilmiştir. Sistem sayesinde müşteri, hesap ve kredi kartı işlemleri gerçek bir senaryo üzerinden modellenmiştir.

# 4. EK KAYNAKLAR

Bu bölümde proje ile ilgili kullanılan ek kaynaklar ve dış bağlantılar yer almaktadır. Projenin kaynak kodları GitHub üzerinde paylaşılmış, ayrıca proje tanıtımı YouTube videosu ile desteklenmiştir.

GitHub Proje Kodu:

https://github.com/matinhaqeat/NYP_24360859224_AbdulMatinHAQEQAT

YouTube Proje Videosu:
https://youtu.be/PplxQKgngXI

# 4. KAYNAKÇA


- Bursa Teknik Üniversitesi ders notları
- Oracle Java Documentation
- Object-Oriented Programming kaynakları